몬스터 관리 5가지 방법

큰 맵에 몬스터를 흩뿌려 놓으면 누구를 켜두고 누구를 끌 것인가

로스트아크처럼 하나의 거대한 맵에 스포너가 여기저기 흩어져 있는 게임 기준

맵
끊기지 않는 하나의 큰 야외 맵

몬스터
스포너가 맵 곳곳에 산재. 전부 켜면 무겁다

화면
쿼터뷰라 한 번에 보이는 땅은 25m 정도

정할 것은 딱 두 가지입니다.

1. **누구를 존재하게 할까** — 몬스터를 아예 만들지 말지 결정합니다. **방법 1~4**가 전부 이 질문에 대한 답입니다.
2. **얼마나 자주 돌릴까** — 만들긴 하되 생각하는 횟수를 줄입니다. **방법 5**가 이쪽입니다.

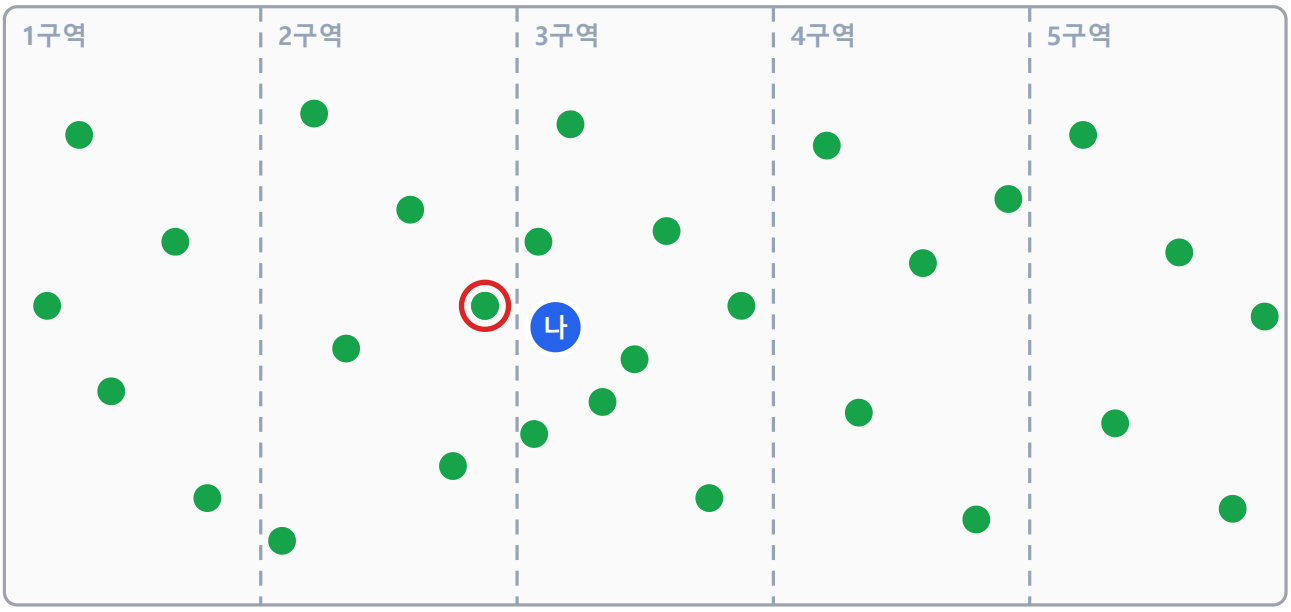
이 둘은 서로 다른 문제입니다. 그래서 마지막에는 하나를 고르는 게 아니라 둘을 겹쳐 쓰게 됩니다.

읽기 전에. 이 문서의 비교는 **아직 측정 전의 예상**입니다. 5가지를 전부 구현해서 같은 기준으로 잰 뒤 이 문서를 실측으로 다시 채웁니다. 현재 실제로 만들어져 있는 것은 **방법 1과 2**뿐입니다.

방법 1 전부 켜두기 관리 없음

맵에 있는 몬스터를 처음부터 끝까지 전부 살려둔다.

돌아가는 몬스터 29마리 / 전체 29마리



완전 가동

어떻게 하는 거냐면

아무것도 안 합니다. 게임이 시작되면 모든 스포너가 몬스터를 뽑고, 그 몬스터들은 게임이 끝날 때까지 계속 생각하고 움직입니다.

좋은 점

- 코드가 필요 없다. 버그가 생길 구석이 없다.
- 이상한 일이 절대 안 생긴다. 몬스터가 사라지거나 갑자기 나타나지 않는다.
- 몬스터가 몇십 마리뿐인 작은 게임이면 **이게 정답이다.**

나쁜 점

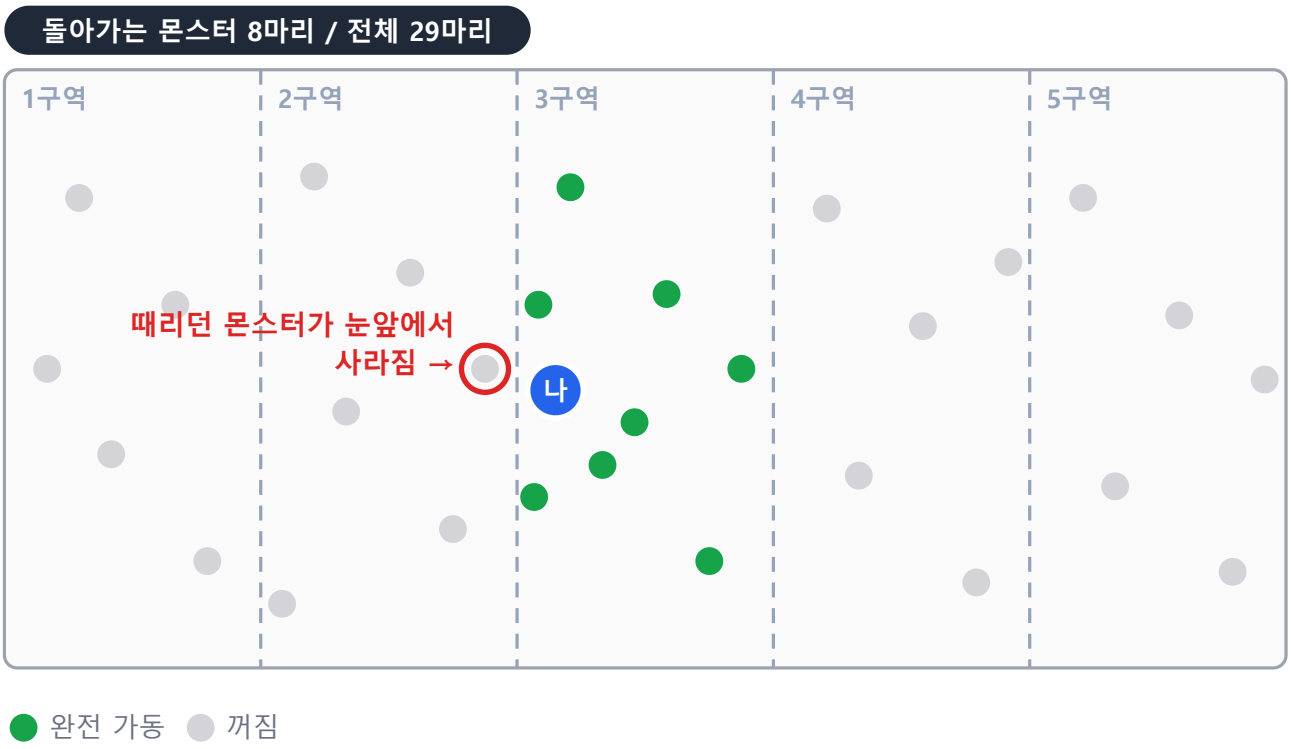
- 몬스터가 늘어나는 만큼 그대로 무거워진다.
- 화면 반대편, 1km 밖 몬스터도 열심히 생각하고 있다. 전부 낭비다.
- 메모리도 최대로 쓴다.

플레이하면 이런 일이 생깁니다

몬스터가 늘어난 만큼 정직하게 무거워집니다. **몇 마리부터 못 견디는지는 아직 측정하지 않았습니다.** 200 / 1000 / 3000마리로 재서 이 문서를 채울 예정입니다.

방법 2 구역 하나만 켜두기 Zone-based

맵을 구역으로 쪼개고, 내가 서 있는 구역만 켠다. (지금 프로젝트가 쓰는 방식)



어떻게 하는 거냐면

맵에 투명한 상자를 5개 놓습니다. 플레이어가 상자에 들어가면 그 구역 몬스터를 전부 만들고, 나가면 전부 지웁니다.

좋은 점

- 단번에 가벼워진다. 전부 켜기 대비 **실측 ▼57%**. (구역이 5개라고 5분의 1이 되지는 않는다 — 몬스터가 구역마다 고르게 있지 않으니까)
- 만들기 쉽다. 상자 놓고 들어갔는지만 보면 된다.

나쁜 점

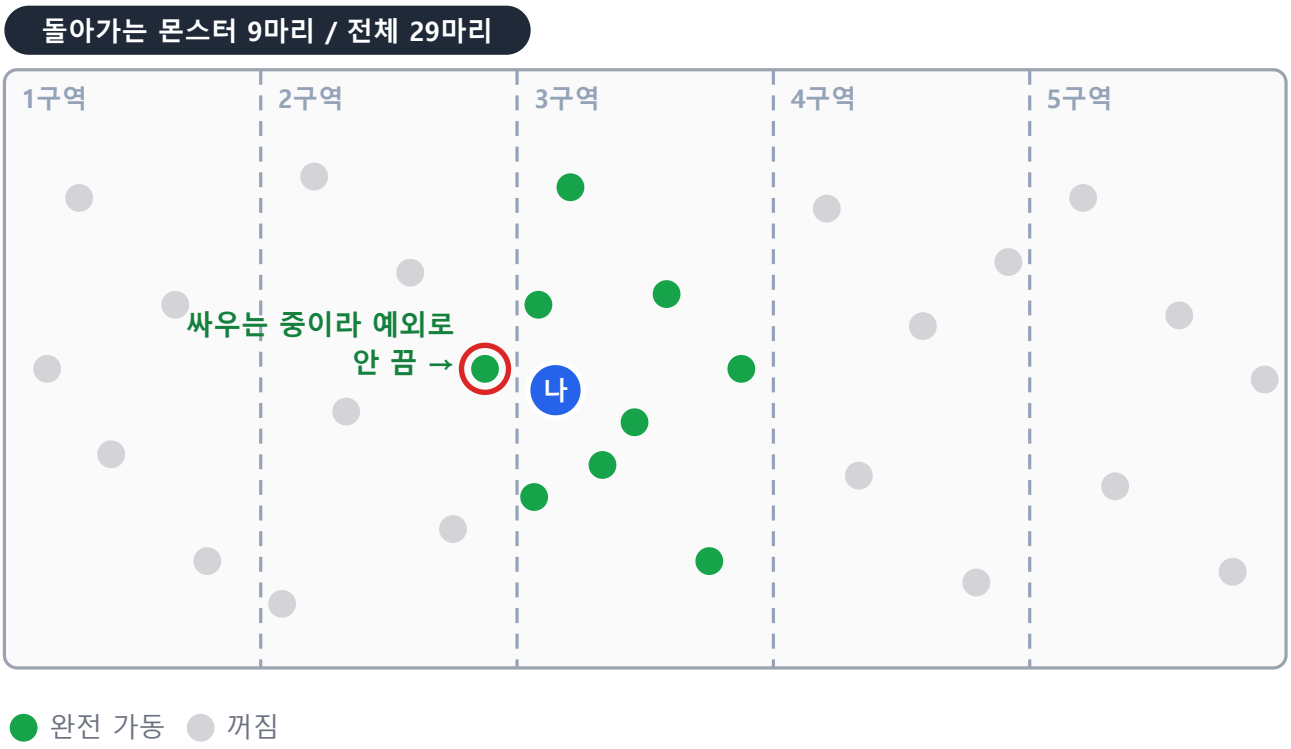
- **경계에서 때리던 몬스터가 사라진다.** 구역을 넘는 순간 이전 구역이 통째로 꺼지니까.
- 구역에 들어가는 순간 몬스터를 한꺼번에 만드느라 화면이 한 번 튄다.
- 구역이 100m인데 화면은 25m다. **안 보이는 몬스터를 잔뜩 켜두고 있다.**
- 껐는데 **메모리는 안 준다.** 풀이 프리팹을 계속 쥐고 있어서.
- 맵이 커지면 구역도 커져서 다시 무거워진다. 근본 해결이 아니다.

플레이하면 이런 일이 생깁니다

몬스터를 절반쯤 깎아놨는데 뒷걸음질 몇 발 쳤더니 그 몬스터가 눈앞에서 뽕 사라집니다. 플레이어 입장에선 그냥 버그입니다. **이 파트의 이야기는 여기서 시작합니다.**

방법 3 구역을 켜되, 싸우는 놈은 예외 Hybrid

방법 2를 그대로 두고 "지금 쓰고 있는 몬스터는 끄지 말자"만 추가한다.



어떻게 하는 거냐면

몬스터가 전투를 시작하면 자기 스포너에 "나 쓰는 중"이라고 등록합니다. 스포너는 쓰는 중인 몬스터가 하나라도 있으면 꺼지기를 거부합니다. 플레이어가 멀어지면 몬스터가 원래 자리로 돌아가고, 그때 등록이 풀리면서 꺼집니다.

좋은 점

- **사라지는 버그가 완전히 없어진다.** 이게 목적이다.
- 기존 구역 구조를 하나도 안 건드려도 된다.
- 여기서 만든 "원래 자리로 돌아가기(리쉬)" 기능은 방법 4에서도 그대로 쓴다.

나쁜 점

- **성능은 조금도 좋아지지 않는다.** 두 구역이 동시에 켜져 있는 시간이 생기니까.
- **최악의 경우 방법 1로 수렴한다.** 경계에 붙어 계속 싸우면 두 구역이 계속 켜져 있다. 그래서 **유예 상한(예: 20초)**을 반드시 걸어야 한다.
- 화면 튐·낭비·메모리는 그대로 남는다. 성능 면에선 아무것도 안 고쳐졌다.
- "쓰는 중" 등록을 한 번이라도 놓치면 그 구역이 영원히 안 꺼진다. 조용히 새는 구조라 위험하다.

플레이하면 이런 일이 생깁니다

버그는 사라집니다. 그런데 성능은 하나도 안 좋아집니다. **증상만 막았지 원인은 그대로라는 걸 확인하는 단계입니다.** 이 실패가 "구역이라는 틀 자체를 바꿔야 한다"는 근거가 됩니다.

방법 4 내 주변 거리로만 판단 Aura-based

구역이라는 개념을 버리고, 내 반경 몇 미터 안에 있는 것만 켠다.



어떻게 하는 거냐면

플레이어를 중심으로 원을 그립니다. 원 안에 들어온 스포너는 몬스터를 만들고, 원 밖으로 나간 스포너는 지웁니다. 구역도, 경계도, 옆 구역이 뭔지도 알 필요가 없습니다.

좋은 점

- **경계라는 게 아예 없으니 경계 버그도 없다.** 고친 게 아니라 없앤 것.
- 화면에 보이는 만큼만 켠다. 낭비가 최소.
- **맵이 아무리 커져도 켜는 개수가 안 늘어난다.** 내 주변 밀도만 중요하니까. 방법 2가 "몇 분의 1"로 줄이는 것이라면 이건 아예 안 늘어나는 것.
- 안 쓰는 몬스터를 진짜로 지우니 **메모리를 줄일 길이 열린다.**

나쁜 점

- 원 경계에서 왔다갔다하면 몬스터가 깜빡인다. (들어올 때와 나갈 때 기준을 다르게 줘서 막는다)
- 멀리 있던 몬스터가 가까이 오면 **없다가 툭 생긴다.** 팝인 현상.
- **몬스터가 한곳에 몰리면 무용지물이다.** 다 원 안에 들어와 버려서 걸러낼 게 없다.
- **이것만 해서는 메모리가 안 준다.** 지워도 풀이 프리팹을 계속 쥐고 있어서, 풀·핸들 반납 정책을 같이 고쳐야 실제로 MB가 떨어진다.

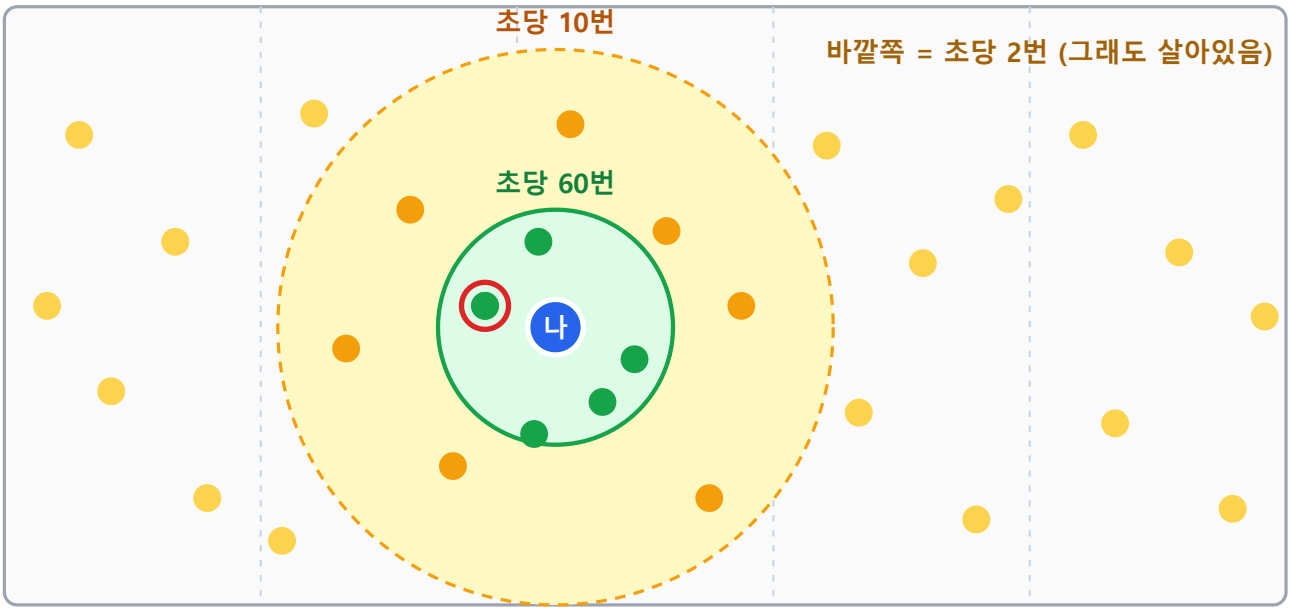
플레이하면 이런 일이 생깁니다

돌아다니는 플레이에서는 압도적으로 가볍습니다. 그런데 몬스터 30마리를 한곳으로 유인해서 몰이하면, 30마리가 다 원 안에 있으니 방법 1과 똑같아집니다.

방법 5 끄지 않고, 대신 느리게 Significance / LOD

아무도 지우지 않는다. 멀리 있는 몬스터는 생각하는 횟수만 줄인다.

꺼진 몬스터 0마리 — 29마리 전부 돎 (속도만 다름)



초당 60번 5 초당 10번 7 초당 2번 17

어떻게 하는 거냐면

몬스터를 거리에 따라 세 등급으로 나눕니다. 가까우면 1초에 60번 생각하고, 중간이면 10번, 멀면 2번만 생각합니다. 몸은 계속 매 프레임 부드럽게 움직이고 **생각만** 가끔 합니다. 그래서 느려진 게 눈에 안 보입니다.

좋은 점

- **아무도 안 사라지고 아무도 안 튀어나온다.** 팝인이 원천적으로 없다.
- **몰려 있어도 효과가 있다.** 30마리가 다 가까이 있어도 등급을 나눠 깎을 수 있다.
- 방법 4와 방향이 다르니 **둘을 같이 쓸 수 있다.**
- 언리얼 엔진이 밀고 있는 방식이 바로 이것이다. (Mass AI, Significance Manager)

나쁜 점

- **메모리는 1바이트도 안 준다.** 아무도 안 지우니까.
- **맵이 커지는 것에는 약하다.** 맵이 10배가 되면 초당 2번짜리 몬스터도 10배가 된다. 방법 1보다 훨씬 완만할 뿐, **여전히 몬스터 수만큼 무거워진다.**
- 멀리 있는 몬스터의 반응이 둔해진다. 가까워질 때 즉시 등급을 올려줘야 한다.
- 몬스터가 저마다 알아서 Update를 도는 구조를 먼저 걷어내야 해서 밑작업이 필요하다.

플레이하면 이런 일이 생깁니다

몬스터가 사라지는 일도 튀어나오는 일도 없이 프레임이 좋아집니다. 대신 메모리는 그대로고 맵이 커지면 다시 무거워지니, **방법 4와 겹쳐 써야** 둘 다 막힙니다.

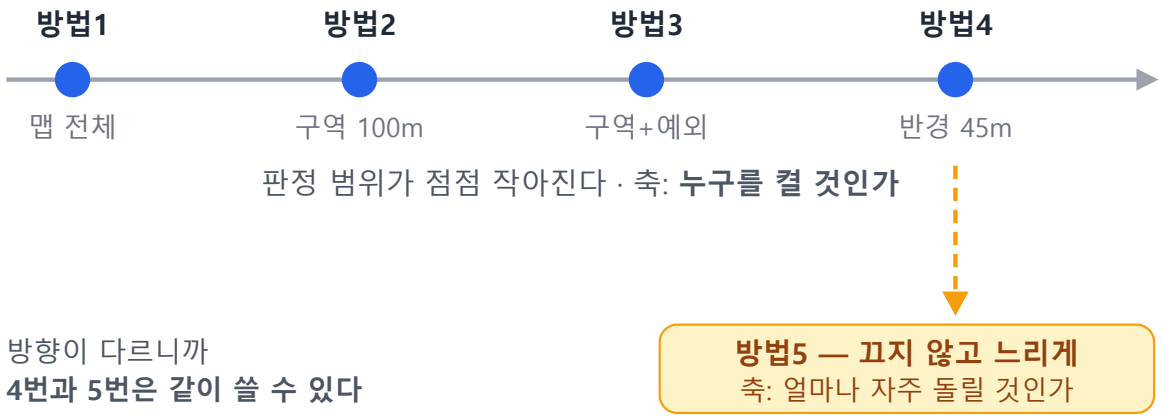
한 장으로 비교

칸마다 이기는 방법이 다릅니다. 그래서 "무조건 이게 최고"가 아니라 "내 게임은 어떤 칸이 중요한가"로 고르게 됩니다.

> **이 표는 아직 측정 전의 예상입니다.** 구조를 보고 추론한 것이고, 잰 값이 아닙니다. 5가지를 전부 구현한 뒤 같은 기준으로 재서 이 표를 다시 채우고, **예상이 맞았는지 틀렸는지까지** 기록합니다.

	1 전부 켜기	2 구역	3 구역+예외	4 거리	5 느리게
경계에서 때리던 몬스터가 사라짐	안 생김	**생김**	안 생김	안 생김	안 생김
없다가 갑자기 튀어나옴(팝인)	없음	**심함**	**심함**	있음	**없음**
쭉 돌아다닐 때 가벼움	**나쁨**	보통	보통	**최고**	좋음
몬스터가 한곳에 몰렸을 때	**나쁨**	**못 거름**	**못 거름**	**무용지물**	**유일하게 효과**
맵이 10배 커지면	**무너짐**	**다시 무너짐**	**다시 무너짐**	**영향 없음**	**완만해질 뿐 계속 늘어남**
메모리가 줄어드나	**안 줌**	**안 줌**	**안 줌**	**줄일 수 있음** 단, 풀 반납도 같이 고쳐야	**안 줌**
만드는 난이도	**없음**	쉬움	보통	보통	어려움

방법 1~4는 한 줄 위에 있고, 방법 5는 방향이 다르다



검토했지만 만들지 않은 것 — "옆 구역도 같이 켜기"

방법 2를 보면 제일 먼저 떠오르는 개선입니다. 그런데 **구역이 100m인데 화면은 25m**라, 옆 구역까지 켜면 안 보이는 몬스터를 2~3배로 늘릴 뿐입니다. 그리고 "어느 구역이 이웃인지"를 손으로 배선해야 해서, 구역을 옮기거나 추가할 때마다 조용히 틀려집니다. 그래서 구현하지 않고 여기 기록만 남깁니다.

그래서 결론

1. **방법 1은 기준선입니다.** 가장 무겁지만 버그가 없습니다. 다른 방법이 얼마나 좋아졌는지 재려면 이게 있어야 합니다.
2. **방법 2는 지금 상태이고, 버그가 있습니다.** 가볍긴 한데 경계에서 몬스터가 사라집니다.
3. **방법 3은 일부러 실패하는 단계입니다.** 버그는 고쳐지지만 성능이 되돌아갑니다. **구역이라는 틀 안에서는 두 마리 토끼를 못 잡는다**는 걸 숫자로 보여주는 게 이 방법의 역할입니다. 그래서 틀을 바꿔야 한다는 결론이 나옵니다.
4. **방법 4가 틀을 바꿉니다.** 구역을 버리고 거리로 갑니다. 경계 자체가 없어지고, 맵이 커져도 안 무너집니다. 대신 몰려 있으면 못 걸러내고, 팝인이 생깁니다.
5. **방법 5는 4가 못 하는 칸을 채웁니다.** 끄는 게 아니라 느리게 하는 것이라 방향이 다르고, 그래서 **4와 같이 쓸 수 있습니다.**

최종 답

방법 4로 **누가 존재할지**를 정하고, 방법 5로 **존재하는 것들을 얼마나 자주 돌릴지**를 정합니다. 둘은 경쟁 관계가 아니라 층이 다른 겁니다.

방법 1~3은 버리는 게 아닙니다. 이 결론이 왜 필요했는지를 증명하는 과정이고, 포트폴리오에서는 **결론보다 이 과정이 더 중요합니다.**

참고한 자료

Interest Management for Distributed Virtual Environments: A Survey — ACM Computing Surveys · 방법 2 = Zone-based, 방법 4 = Aura-based 라는 정식 분류명이 여기서 나옵니다

Significance Manager — Unreal Engine 공식 문서 · 방법 5의 원형. "트리거 볼륨에 의존할 수 없는 비선형 게임에 특히 유용하다"고 명시되어 있습니다

Mass AI 크라우드 LOD 4단계 — StraySpark 기술 블로그 · 1단계가 0~25m로, 우리 화면 크기와 일치합니다 (스튜디오 블로그 벤치마크)

CullingGroup API — Unity 공식 매뉴얼 · 방법 4의 구현 수단